26. Load Balancing Concepts and Health Checks

26. Load Balancing Concepts and Health Checks

Phase: Phase 10 — Load balancing and proxies
Estimated time: 1–2 days
Slug: `26-load-balancing`

Learning objectives

- Explain L4 vs L7 balancing
- Know health checks and draining
- Recognize algorithms (round-robin, least-conn)
- See nginx upstream basics

Prerequisites

Complete prior lessons. Prefer a Linux VM for `ip`/`ss`/`nft` examples; macOS works for many `curl`/`dig` drills. Type every command yourself and keep a lab notes file.

Why this matters

One backend is sick but still in pool; 1/N users fail. Health checks, idle timeouts, and L4 vs L7 balancing decide whether you drain or page.

Core concepts

What you must be able to explain:

- Explain L4 vs L7 balancing
- Know health checks and draining
- Recognize algorithms (round-robin, least-conn)
- See nginx upstream basics

Mental model for `26-load-balancing` (exercise kind `lb_l4l7`):

- Prefer **measurable** checks (exit codes, ICMP replies, DNS rcode, HTTP status, socket state) over vibes.
- Separate **reachability** (can packets flow?) from **correctness** (right service, cert, route, identity).
- Ask: *which layer's tool falsifies my current hypothesis?* Then run that tool.
- Record the exact command that proved the root cause — that becomes your playbook snippet.

Worked commands

Run these on a lab host. Linux VM preferred for `ip`/`ss`/`tcpdump`/`nft`; macOS is fine for many `curl`/`dig`/`ping` drills. Your addresses and timings will differ — match the *shape* of the output and the conclusion you draw.

Example 1

```
curl -I https://example.com 2>&1 | head -15
```

Sample output:

```
HTTP/2 200
content-type: text/html
cache-control: max-age=...
```

Interpretation: Response line and headers are L7 evidence after TCP/TLS succeeded.

Example 2

```
curl -sS -o /dev/null -w 'code=%{http_code} ttfb=%{time_starttransfer}
total=%{time_total} ver=%{http_version}\n' https://example.com
```

Sample output:

```
code=200 ttfb=0.120 total=0.180 ver=2
```

Interpretation: Write-out metrics separate connect delay from first-byte and total time — essential for gateway/CDN debates.

Example 3

```
echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/
null | openssl x509 -noout -subject -dates
```

Sample output:

```
subject=CN = example.com
notBefore=...
notAfter=...
```

Interpretation: Certificate identity and expiry are independent of HTTP 200. SNI (`-servername`) selects the cert on shared IPs.

Mini-lab

1. Re-type every worked command for **Load Balancing Concepts and Health Checks** and paste outputs into lab notes.
2. Introduce one controlled failure in the lab (wrong DNS, firewall drop, bad route) and recover using only this lesson's tools.
3. Write a 5-line playbook: symptom → first command → expected healthy output.
4. Complete the exercise stubs; then explain each function's return value without looking.

Failure modes

Symptom	Likely cause	Next check
No health checks	See common mistakes	Re-run lesson commands
Balancing without shared session stra…	See common mistakes	Re-run lesson commands
Draining ignored during deploys	See common mistakes	Re-run lesson commands
Timeout with no RST	Silent drop / blackhole	tcpdump + compare path

Study tip for Load Balancing Concepts and Health Checks

Before each new command, write a one-line hypothesis. If the result falsifies it, update the hypothesis instead of stacking random tools. Note equivalents across OS (`ip` ↔ `ifconfig`/`route`, `ss` ↔ `netstat`). Re-run two commands from this lesson tomorrow from memory and explain the sample outputs out loud.

Evidence habit

When pages are down, prefer evidence (`ip route get`, resolver answer, packet capture, listener list) over intuition. Document **command → output → conclusion** in a lab notebook so the same path is reusable on-call. Change one variable at a time; write a one-line hypothesis before each new tool.

Common mistakes

- No health checks
- Balancing without shared session strategy when needed
- Draining ignored during deploys

Exercises

Work in `exercises/26-load-balancing/`. Read that folder's `README.md` before coding.

```
cd exercises/26-load-balancing
chmod +x test.sh exercise.sh 2>/dev/null || true
./test.sh
```

What you will build

Edit `exercise.sh` and implement `http_path_balancing_layer`, `port_only_balancing_layer`, `backend_probe_name`. Each function is a tiny retrieval drill for **Load Balancing Concepts and Health Checks** — return the exact tokens/flags/labels from the lesson. Keep the comment above each function describing when you would use that fact on-call. Run `./test.sh` until all 3 assertions pass.

Study loop

- **A (Drill):** make tests pass
- **B (Apply):** run 2–3 real commands from this lesson in your lab and note outputs
- **C (Challenge):** rewrite answers from memory next day

Do not open `solutions/26-load-balancing/` until after an honest attempt.

Self-check

Answer without scrolling up:

1. L4 vs L7?
2. Why health checks?
3. What is draining?

Further reading

- https://nginx.org/en/docs/http/load_balancing.html